

Rapport IFT2015 - TP1

Nom: Ryan Ramaherison Mac Way Kit - Matricule: 20306738

Nom: Arnaud Mehrabi - Matricule: 20302443

1. Auto-évaluation

Notre programme `Tp1.java` fonctionne: **Correctement**. Il contient 3 algorithmes de tri, à savoir : **Tri par Insertion** (Option 1), **Tri par Fusion** (Option 2) et **Tri par Arrays.sort()** (Option 3)

2. Analyse de la complexité temporelle (pire cas) théorique

L'analyse asymptotique ci-dessous détaille le comportement algorithmique théorique en fonction de la taille du problème n (nombre de bâtiments), représentant le nombre total de données à trier.

Algorithme de Tri	Complexité (Pire Cas)	Justification Théorique
Tri par Insertion (Insertion Sort)	$O(n^2)$	Algorithme qui contient 2 boucles imbriquées. La boucle externe <code>for</code> s'exécute linéairement $n-1$ fois, proportionnelle à $O(n)$. Et la boucle interne <code>while</code> qui s'exécute au max i fois, proportionnelle à $O(i)$ (où i ne peut pas dépasser n). Puisque ce sont deux boucles imbriquées, les complexités se multiplient, d'où $O(n^2)$ en pire cas et en cas moyen (cette méthode est, en meilleur cas, $O(n)$ quand le tableau est déjà trié). Ce tri possède une complexité spatiale $O(1)$ car elle ne crée aucune structure de donnée pour trier le tableau.
Tri par Fusion (Merge Sort)	$O(n \log n)$	Algorithme de type "Diviser pour régner". Le tableau de données est divisé récursivement en 2 sous-tableaux équilibrés jusqu'à atteindre des singletons. La profondeur maximale de cette récursion est de l'ordre de $\log_2 n$. À chaque niveau de cette récursion, la méthode de fusion linéaire <code>merge()</code> combine les sous-tableaux triés en un seul tableau trié, proportionnel à $O(n)$. Puisque cet algorithme est récursif, les complexités se multiplient ($\log_2 n$ divisions en pire cas multipliées par n étapes de reconstitution du tableau). D'où une complexité temporelle finale de $O(n \log n)$ en pire, moyen et meilleur cas. Sa complexité spatiale est $O(n)$ à cause de la création de la variable <code>temp</code> .
Tri par Arrays.sort()	$O(n \ln(n))$	Algorithme natif de Java qui fait appel à la méthode de la librairie standard <code>Arrays.sort()</code> pour les tableaux dynamiques et réalise automatiquement un algorithme le plus adapté au tableau de données. Puisque les méthodes de tri n'agissent pas sur des tableaux contenant des types primitifs de Java, <code>Arrays.sort()</code> applique le Timsort, de complexité temporelle $O(n \ln(n))$ en pire cas. Cet algorithme procède en parcourant le tableau à la recherche de cas de suites d'au moins deux éléments consécutifs croissants puis, lorsque ces éléments ne sont pas alignés, effectuer un tri par insertion pour ordonner plusieurs éléments consécutifs. Une pile est tenue pour garder en mémoire de telles sections du tableau, que l'algorithme utilise pour trier des sections de manière successive. Ce tri se fait de manière similaire au merge sort (l'algorithme trie les sections par paires jusqu'à ce que l'entièreté du tableau soit trié).

3. Analyse empirique de la complexité temporelle

Les temps d'exécution de chacun des algorithmes constatés ci-dessous ont été enregistrés à l'aide de la méthode `System.currentTimeMillis()` de Java dans le programme `Tp1.java` dont les fichiers d'entrée utilisés pour faire les tests sont situés dans le répertoire `./exemples`. Comme nous pourrions nous en douter, l'algorithme simple est plus lent que les deux autres à cause d'un plus grand nombre d'étapes à franchir lors de son exécution (bien que cette différence ne se fasse pas ressentir lorsque le nombre de points de cargaisons est petit).

